课程实践　·　大作业一

第二阶段项目报告

需求变更迭代　·　基于 MongoDB 的在线问卷系统

姓　　名　　邱浩峰

提交日期　　2026 年 10 月 29 日

技术栈　　Node.js • Express • Mongoose • MongoDB

交付内容　　需求变更分析 • 数据结构演进 • 可运行实现 • 自动化测试

测试规模　　92 个自动化用例全部通过（第二阶段新增 23 条）

运行方式　　npm install → npm run seed → npm start

2026 年 10 月

一、需求变更与分析

系统在第一阶段交付并投入使用后，收到八项需求变更。八条反馈看似零散，指向同一个结论：**题目应当脱离问卷独立管理**。

表 1-1　八项需求变更

#	用户反馈	期望效果
1	每次做类似问卷都要重写题目	可存常用题；建卷时直接选用；一题可被多卷使用
2	团队协作只能复制粘贴	可把题共享给他人；对方能在自己问卷里使用
3	改了题，以前问卷里的题也跟着变	已发布问卷的题不可改；旧问卷保持原内容
4	改完想找回旧版本、想知道题从哪改来	可看历史版本、可恢复旧版本
5	同一题想小改一下用在别的问卷	可基于一题生成新版本，各问卷用不同版本
6	想改题但不知道哪些问卷在用它	可查题目被哪些问卷引用
7	希望把常用题整理成题库	可管理题库、加入题库、从题库选题
8	想看一道题在**所有**问卷中的回答	单题跨问卷统计

按风险归类，这八条其实是两类问题。需求 3、4、5 是"改题要不要影响历史"的三个侧面——只要问卷侧不再依赖题库的可变状态，三条一起解决；需求 1、2、6、7、8 是"题目的作用域从一份问卷扩大到多份"之后衍生的查询与权限需求。这个判断决定了后面只做两个数据结构决策，而不是八个。

回顾第一阶段，两条既有设计直接影响了本次改动。**其一是题目独立成集合：**`questions` 每道题一个文档，问卷只存 `question_count` 冗余计数，当时这样做是为了编辑期并发安全，结果让"把题挂到题库上"不需要动问卷文档结构。**其二是发布后冻结题目结构：**`draft → published → closed` 是单向状态机，所有增删改入口都有 `assertDraft` 守卫——需求 3 在

第一阶段**已经由这条规则完整满足**，本次没有为它写业务代码，只补了一条回归用例（T82）防止"从题库加题"这条新通道绕过守卫。真正的新问题是：第一阶段没有"一道题属于多份问卷"和"同一题多版本"的表示方式，而引入新结构时最大的风险恰恰是需求 3——若处理不当，题库的"更新"会反过来破坏问卷快照。所以本次设计的第一原则不是"怎么复用"，而是"怎么复用才不破坏历史"。

二、三个核心决策

2.1 决策一：问卷侧存"快照"，不是"引用"

备选方案：问卷题只存 `bank_question_id`，读取时 join 题库最新版本。

否决理由：这正是需求 3 要禁止的行为。题库一更新，所有已发布问卷的题目内容随之变化，历史答卷的题干与选项立即失去解释依据——"去年那批人选的'非常满意'到底是五个选项中的哪一个"再也答不上来。问题不在代码写错，而在问卷题与题库题共享了同一份可变状态。

最终方案：从题库选题时，把题目内容（题干、选项、约束）整体拷贝为问卷自己的题目文档；`bank_question_id` 与 `bank_version` 只作溯源字段，记录"来自题库哪个文档的哪个版本"。此后题库无论怎么出新版本，问卷快照一个字节都不变。需求 3（旧问卷不变）与需求 5（不同问卷用不同版本）由这一个决策同时满足，且不依赖任何额外同步或后台任务。

代价是存储冗余：一道题被 100 份问卷使用就有 100 份拷贝。这是刻意取舍——题目内容只有几百字节，而"历史答卷可解释"是无价的；快照本来就是"当时问的是什么"的唯一权威记录，它不是缓存，不该被优化掉。

2.2 决策二：题库题"版本即文档"

修改题库题**永远不原地覆盖**，而是插入新文档：`version + 1`，`parent_id` 指向修改前的

版本，同一棵版本树的文档共享 `root_id`（v1 的根是它自己）。

对比被否决的"数组内嵌 history"方案：内嵌数组随修改次数无限增长，逼近 16 MB 文档上限；并发修改会在数组层面互相覆盖；更关键的是数组里的历史版本无法被分别引用——而需求 5 要求不同问卷分别用 v1 和 v2，只有独立文档才有自己的 `_id` 可供问卷溯源字段指向。

这个结构下需求 4 的三个动作都有自然实现：看历史 = 按 `{root_id, version}` 列出版本链；知道来源 = 读 `parent_id`；恢复旧版本 = 以旧版本内容在树末端追加一个新版本，历史一条不删。恢复因此是"再改一次"而不是"回滚"，顺带避免了"回滚后引用旧版本的问卷指向虚空"的问题。

2.3 决策三：共享是"树级"语义

`shared_with`（用户 id 数组）内嵌在题库题文档上，共享与取消共享对整棵版本树 `updateMany`。有两个容易漏掉的细节：其一，作者共享 v1 后又改出 v2，若 v2 不继承 `shared_with`，被共享者会"丢题"，所以新版本文档创建时显式继承来源版本的共享列表；其二，被共享者的权限是"读 + 选题进问卷 + fork"，**不能**修改原题（403），团队协作由 fork 收尾——fork 生成一棵 owner 是自己的新树，此后各改各的，从根上避免写写冲突，也不需要锁或冲突合并。

2.4 方案的代价与边界

把边界写清楚比宣称"完美"更有用。**版本树只增不删**：一题改 20 次就有 20 个文档，当前用索引支撑"取最新版"、列表只展示最新版本以压住视觉噪音，长期运行需要归档策略。**不做分支合并**：fork 出的新树与源树再无关联，因为需求要的是"各改各的"，Git 式三方合并对普通用户过重。**共享粒度是整棵树**，不能只共享 v2 不共享 v1——被共享者需要看到版本演进才敢用。**快照不追溯**：题库出 v2 后，已引用 v1 的问卷不会自动升级，要升级只能

由作者在草稿状态重新选题，这条限制正是需求 3 的保证。

三、数据结构变化与兼容性

3.1 新增集合 bank_questions

题库题独立成集合，而不是在 `questions` 上加一堆可空字段，原因是两者生命周期完全不同：问卷题随问卷发布而冻结，题库题要被反复修改、共享、fork。

表 3-1 bank_questions 集合字段设计

字段	类型	说明
`owner_id`	ObjectId → users	创建者；决定修改、共享、恢复权限
`type`	String 枚举	七种题型，与 questions 一致
`title` / `required` / `options` / `constraints`	同 questions	题目内容本体
`version`	Number ⩾ 1	树内版本号，从 1 开始
`root_id`	ObjectId → bank_questions	版本树根（v1 的 `_id`）
`parent_id`	ObjectId / null	修改或 fork 的来源版本；null 表示原创 v1
`shared_with`	[ObjectId]	被共享用户列表（树级语义）

3.2 questions 新增溯源字段与索引

表 3-2 questions 新增字段与新增索引

新增项	类型 / 键	作用
`bank_question_id`	ObjectId / null	溯源：来自题库哪个文档；非题库题为 null
`bank_version`	Number / null	选题时的版本号快照
索引 `{owner_id:1, updated_at:-1}`	bank_questions	我的题库列表（按更新时间倒序）

新增项	类型 / 键	作用
索引 `{root_id:1, version:-1}`	bank_questions	版本树查询、取每棵树最新版
索引 `{shared_with:1}`	bank_questions	"共享给我的"反查
索引 `{bank_question_id:1}`	questions	使用情况反查、跨问卷统计定位

题目内容本体不变——快照在加入问卷那一刻已完整拷贝，两个溯源字段不参与任何渲染或校验计算，因此旧问卷题（两字段为 null）不需要走特殊代码路径。

3.3 兼容性

旧数据零迁移： `bank_question_id` / `bank_version` 均可空，第一阶段创建的问卷题两字段为 null，读取端按"非题库题"分支处理，问卷、答卷、统计全部原样可用。**答卷零影响：** `responses.answers` 通过 `question_id`（问卷快照的 id）关联，与题库完全解耦。**接口向后兼容：** `POST /surveys/:id/questions` 原有请求体行为不变，`bank_question_id` 是新增可选字段。**失败关闭：** 若 `bank_question_id` 指向不存在或无权访问的题，接口返回 400/404 而非静默忽略——加题失败必须让用户知道，否则会得到一份少题的问卷。

四、接口设计与实现要点

新增 `routes/bank.js`，挂载于 `/api/bank-questions`，11 个端点覆盖八项需求。

表 4-1 题库接口与需求对应关系

需求	端点
创建 / 列表 / 详情	`POST /`、`GET /?scope=mine|shared`、`GET /:id`
修改（= 新版本）	`PUT /:id`（部分更新：未传字段继承当前版本）
多版本并存 / 接收共享	`POST /:id/fork`
版本历史 / 恢复旧版本	`GET /:id/versions`、`POST /:id/revert`

需求	端点
共享 / 取消共享	`POST /:id/share`、`POST /:id/unshare`
使用情况 / 跨问卷统计	`GET /:id/usage`、`GET /:id/stats`

实现上有三点值得记录。**权限三级模型**：owner（可改、共享、恢复）、被共享者（可读、选题、fork）、其他人（403 而非 404）——题目 id 本身不是敏感信息，明确告知"无权访问"比伪装"不存在"更好排障，真正敏感的问卷标题与答卷内容另有更严格的边界（见 §6.3）；所有端点入口统一走 `loadAccessibleBankQuestion()`，避免每个 handler 各写一遍判断。**跨问卷统计是一条聚合管道**：先按溯源字段定位该题所有版本在所有问卷中的快照，再 `$match(survey_id) → $unwind(answers) → $match(question_id)`，然后按题型分支——选项类 `$unwind option_keys` 后 `$group` 计数，数字类取 `$min/$max/$avg`，文本类 `$push` 后 `$slice` 最近 20 条，计算全部下沉到数据库侧。**开发中修掉的两个自身设计错误**：① `root_id` 一度设为 `required`，但 v1 需要先插入拿到 `_id` 才能回填，`create()` 在插入瞬间就跑校验导致必然 400，改为去掉 required、由写路径约定保证非空；② 修改接口初稿是全量语义，只传题干会把选项清空，改为部分更新。

五、前端页面与界面优化

题库入口加入全站导航，题库做成与"我的问卷"平级的独立页面，而不是塞进编辑页的侧栏。理由是两者心智模型不同——问卷编辑是"往下写"，题库管理是"横向整理"，混在一起会让"我现在改的是这份问卷还是那道公共题"变得含糊；而选题动作又必须在编辑页完成，所以编辑页保留"从题库选题"入口，点开是带"我的 / 共享给我的"两个页签的弹窗。"独立管理 + 就地选用"是这次前端设计里唯一一次真正的结构选择。



图 5-1　题库管理页：每棵树只显示最新版本，操作按主次分组



图 5-2　问卷编辑页（草稿）：题目列表与"更多"菜单

功能完成后，我对全部六个页面做了一轮界面整理，参照主流问卷系统（问卷星 / 金数据 / 腾讯问卷）的信息架构，补齐了几处原本缺失的东西。**首页加指标区**：新增 `GET /surveys/overview` 一次返回各状态问卷数、累计答卷与题库题数，避免前端为每份问卷各请求一次统计；列表补标题搜索（服务端正则匹配，输入做正则转义）、状态分段筛选与排序。**列表页操作分主次**：原来六个按钮平铺，标题被挤到换行；改为"编辑 / 统计"两个主操作加"更多"菜单（分享与二维码、复制为草稿、导出、删除），问卷标题恢复单行。**编辑页先说状态再做操作**：草稿 / 已发布 / 已关闭各有一条状态横幅说明"现在能改什么"，问卷设置收进折叠面板，题目行压缩为"序号 + 题干 + 题型标签 + 主次操作"，一屏可见的题目从 3 道增加到 9 道；已发布时增删题入口隐藏，与后端 `assertDraft` 形成两道独立防线。**统计页两列布局**：选项与数字题并排，一屏可见题目从 2 道增加到 6 道；数字题补一条"均值在区间中的位置"刻度条，比只列最小/最大更容易判断分布偏哪边；明细不再显示数据库主键。**填写页**：进度条带百分比与题号方格，新增"上一题"（跳转规则下按访问栈回退，答案保留），选项改为整块可点的卡片。所有改动沿用像素风（硬边框、位移硬阴影、方块图标），新增的是组件而非换皮。

六、多用户可用性优化

八项需求完成后，我做了一轮针对"别人来用会不会卡住"的审计。出发点很具体：这个系统不是给我一个人演示用的，第二个注册的用户、被共享题目的同事、以及手滑改坏 URL 的人，都会真实碰到它。

6.1 审计与修复

不读代码找 bug，而是代入三个角色各走一遍完整流程：第二个注册的用户（登录后看到什么？空状态有没有引导？会不会看到别人的数据？）、被共享题目的同事（能做什么、不该看到什么？）、手滑的用户（少个参数、id 被改坏、问卷已关闭还点保存）。共修复 11 处，

按影响排序如下。

表 6-1　可用性审计修复清单

#	问题	影响	修复
1	列表页用了 `toast(...)` 但解构时漏了 `toast`	复制问卷成功后抛 `ReferenceError`，列表不刷新	补进解构列表
2	路径参数非 ObjectId 时抛 `CastError`	直接 500"服务器内部错误"，用户不知找谁	全局错误处理转为 400
3	题库用量 / 统计对被共享者开放	共享一道题就泄漏对方问卷标题与答卷汇总	收敛为仅 owner（403），前端隐藏按钮
4	改题库题时顺手改了旧版本 `updated_at`	"历史版本"会被后续操作改写，语义被破坏	只追加新版本，不写旧文档
5	登录页对 `next` 二次解码	路径含 `%` 时解码错乱；`next=//evil.com` 可跳站外	去掉二次解码，只接受单个 `/` 开头
6	问卷列表固定取 50 条，没有翻页	超过 50 份后更早的问卷彻底看不到	补上一页 / 下一页与页码
7	题库"修改"只能改题干	想改选项或题型只能重建，版本能力形同虚设	改为完整编辑弹窗
8	已关闭的问卷仍显示可点的"保存设置"	点了必然 409，无效操作却无提示	关闭后禁用按钮
9	统计明细显示受访者数据库 ObjectId	对使用者无意义，还暴露内部主键	改为"登录用户 / 匿名填写"
10	反向代理部署时 `share_url` 固定 http	https 站点复制出的链接协议错，扫码打不开	`trust proxy` + `X-Forwarded-Proto`
11	已发布的问卷仍显示"添加题目""从题库选题"	认真填完表单，提交必然 409	发布后隐藏入口，后端守卫仍在

这 11 条里真正会让第二个用户"用不下去"的是前三条：第 1 条是必现的功能断链，第 2 条把用户的手误变成看起来像服务器崩溃，第 3 条是权限边界的实际泄漏——它不是"体验不好"，而是"共享一道题却把一份问卷交了出去"。第 4 条更要紧的是语义：一个叫"历史版本"的东西如果会被后续操作改写，用户对它的信任就是错的。

6.2 双账号验收

自动化测试覆盖接口行为，覆盖不了"两个人真的互相操作时是什么感觉"。因此另起独立数据库与端口，用两个真实账号把完整链路跑了一遍：注册 → 建卷 → 加题 → 存进题库 → 共享给对方 → 对方从共享题库选题 → 出题人改题 → 确认对方问卷快照未变 → 双方分别发布 → 匿名与登录各填一份 → 各自看统计。47 项检查全部通过，其中 6 项越权均如预期返回 403。这一步也顺手验证了"零迁移"：第一阶段的旧问卷在新代码下打开、填写、统计均正常。

6.3 权限边界收敛

审计后明确三条边界并写进测试：**能看到题目**不等于**能看到使用情况**——被共享者拿到的是"一道题"，出题人手里的是"这道题连着哪些问卷、答卷里答了什么"；**能 fork** 不等于**能改原题**；**能填问卷** 不等于**能看统计**。这些判断都在服务端，前端隐藏按钮只是为了不让人点出一个 403。

七、测试

新增 `tests/api.bank.test.js`，T70～T88 共 19 条用例，每条对应一项需求或一条边界；可用性优化后又补了 T89～T92。

表 7-1 第二阶段新增用例与覆盖内容

用例组	覆盖内容
T70 - T72	创建题库题（version=1、root_id=自身）、我的列表、他人隔离与未登录 401
T73 - T75	修改生成新版本、版本链可查、revert 恢复旧版生成 v3 且历史完整
T76 - T78	共享后对方可见、Bob 不能改 Alice 的题（403）但可 fork、越权 403

用例组	覆盖内容
T79 – T80	从题库选题（快照 + 溯源）；**需求 3 核心回归：题库出 v2 后问卷快照仍是 v1 内容**
T81 – T83	使用情况反查、已发布问卷不能再从题库加题（409）、取消共享后不可见
T84 – T88	跨问卷统计完整数据流：两卷引用同题各填答卷后选项计数跨卷累计（2/1/1）、数字题 min/max/avg、未使用题返回全零
T89 – T92	可用性回归：被共享者查用量 403、改题不改写旧版本时间戳、非法 id 返 400、非法 `bank_question_id` 返 400/404

全量结果：**92 条用例全部通过**（第一阶段 69 条回归无破坏 + 第二阶段 23 条新增）。T80 是整套测试里最该保留的一条——它断言的不是某个接口返回什么，而是"改题库之后再读问卷，题干的每一个字都没变"。这条断言一旦失败，说明整个方案的地基塌了。

八、AI 使用记录

本阶段继续使用对话式 AI 编码助手，日志 #21～#26（完整表格见 `docs/ai-usage-log.md`）。

表 8-1　本阶段 AI 使用记录摘要

#	用途	是否修改	说明
21	八条需求分析与方案取舍	修改后采用	AI 初稿建议"问卷引用题库并跟随最新版"，与需求 3 冲突，改为快照 + 溯源
22	题库模型与路由实现	修改后采用	修掉 root_id required 导致的 400、全量更新清空选项两处错误
23	前端题库页与选题弹窗	直接采用	复用像素风类名
24	19 条测试	修改后采用	修掉二次连库挂住、漏 teardown；补上 T80 关键回归
25	跨问卷统计管道	修改后采用	文本样本加 `$slice`

#	用途	是否修改	说明
			限量
26	多用户可用性审计与界面优化	修改后采用	首轮只报代码风格建议，被要求以"第二个用户登录后会发生什么"重跑，才找出 3 个真正让人用不下去的问题

真实对话节选（完整版见 `docs/ai-usage-log.md` §5）：

- **我**：八条需求里最要紧的是"改题不能影响已做好的问卷"。题库题是不是让问卷直接引用，跟着最新版走？
- **AI**：可以让问卷题存 `bank_question_id` 引用题库文档，读取时 join 最新版本。这样复用最直接。
- **我**：不行，这正好违反第三条——老师改了题，去年发布的问卷内容就变了，收集的数据全对不上。
- **AI**：那就反过来：选题时把内容整体拷贝进问卷（快照），`bank_question_id` 只记来源用于反查和统计；题库题修改永远生成新版本不覆盖。这样第三条自动满足，第五条也自动成立。
- **我**：我要的不是代码风格建议。假设我是第二个注册的用户，登录进来点复制问卷，会发生什么？
- **AI**：（重新走查后）复制问卷的回调里调了 `toast(...)`，但页面解构时没取 `toast`，所以副本创建成功、提示抛 `ReferenceError`，列表也不刷新——用户会以为没成功，再点一次。
- **我**：这个才是会让别人用不下去的问题。顺着"用户手滑"再走一遍：地址栏的 id 被改坏呢？另外检查共享后的可见范围。
- **AI**：参数不是合法 ObjectId 时抛 `CastError`，被兜底成 500；`usage` 和 `stats` 目前对所有可见者开放，会连带返回对方问卷标题与汇总的答卷内容。

- **我**：前者换成 400 并说清原因；后者收回，只给 owner。共享的是题，不是问卷。

九、总结

这次需求变更是一次真正的"演进"练习：所有改动都建立在既有代码之上——题目独立集合让题库有了挂载点，发布冻结规则让需求 3 天然满足，统一错误结构与测试基建让 23 条新用例第一天就能跑进既有套件。八项需求最终收敛为两个数据结构决策（快照溯源、版本即文档）和一个权限模型（树级共享），没有推翻任何一张已有的表，旧数据零迁移。

值得注意的是，这次的复用不是"预料到了所以省事"，而是"当时拆得对所以接得上"：`questions` 独立成集合的初衷只是编辑期并发安全，它在第二阶段的用处属于意外收益。而第六节那 11 处修复提供的教训是另一类：**功能做完不等于别人能用**。八项需求全部实现、19 条测试全绿的那天，第二个用户点一下"复制问卷"就会看到一个坏掉的功能；共享一道题，会把一份问卷的统计连带交出去；界面上还摆着一个点了必然失败的加题按钮。这些问题自动化测试抓不到，因为它们不在接口层的正确性里，而在"人怎么用它"里——所以那一轮走查的价值不在于修了 11 个 bug，而在于把"换一个人来用"变成了交付前的固定动作。